数据库系统原理
CHAPTER 1 DBS引论
数据管理的发展
数据
数据：对事物的符号化描述，符号形式可为文
字、图像、音频等等
数据 -> 信息 -> 知识
数据密集型应用：数据量大+持久数据+共享数据
元数据：用于描述数据的数据，如数据的格式，
条目，数量等等
文件系统
通过操作系统进行文件级的读写操作
缺点
数据冗余+不一致性缺乏统一存储
数据独立性差，难维护文件结构修改须同时修改程序
程序编写不便，设计者需了解文件的逻辑与结构缺乏统一接口
数据共享性差，不支持并发访问不能多人cowork
缺少统一管理，安全性差缺乏权限和保护
数据库系统
构成: 应用程序、数据库管理系统DBMS、数据
库DB和数据库管理员DBA
DBA职责
1. 定义DB信息内容和结构
2. 定义DB存储结构和存取策略
3. 定义数据的安全性要求和完整性约束条件。
4. 监控数据库的使用和运行(运维)，进行性能调优。
5. 数据库的备份与恢复。
DBMS功能
高级的用户接口: 数据以逻辑形式展现, 通过非过程数据库语言
(nonprocedural database language)操纵数据库.
非过程化：给出做什么的
命令，无需在意怎么实现
查询(query)处理和优化: 查询泛指各种用户提出的访
问请求（增删改查）, 包括访问\修改\定义等等.
数据目录管理，元数据 & 数据字典DD（存放元
数据的数据库）
并发控制(concurrent control)
访问控制(access control)
完整性约束(integrity constraints)检查: 数据类
型约束\语法检查\语义检查(完整性约束)
什么是数据库的安全性 (Security) 和
完整性 (Integrity)？
恢复功能: 在故障情况下, 也须保持一致性
In another word
为什么说“数据库技术的发展历史就是数据独立
性不断提高的历史”？
数据库技术的发展
第一代：层次（hierarchical）数据库 & 网状（network）数据库
第二代：关系（Relational）数据库
“关系”指是数学中的一个概念——“关系 (Relation)”，也就是“数据
表 (Table)”的学术名称。so 关系 (Relation) = 表 (Table)
第三代：后关系（Post-relational）数据库
数据抽象
抽象设计标准：ANSI-SPARC体系结构
数据模型（Model）
数据模型是描述数据、数据关系、语义和约束的
抽象框架，规定了如何组织和表示数据。
数据的描述
静态特性: 数据的结构\关系\约束
动态特性: 数据上的操作
多级(类)数据模型
概念(conceptual)数据模型: 用来描述一个单位
的概念化结构, 是数据库设计开始阶段的对现实
需求的抽象.
逻辑(logical)数据模型: 用户从数据库中所能看到
的数据模型, 展现了数据间逻辑.
物理(physical)数据模型: 反映数据存储逻辑
SUM: 数据模型是数据库的抽象框架, 分为概念
\逻辑\物理三层, 分别代表对现实的抽象描述\如
何组织具体数据间逻辑\存储逻辑. 三类数据模型
针对数据库设计不同阶段
数据模式（Schema）
数据模式是对数据库的设计，是在某个数据模型下
的具体描述
是运用某种数据模型（手段）对一个企业或组织的
一组相关数据的结构、联系和约束的描述（结果）
数据分为, 型(type)和值(value), 值是型的一个实
例. 型的描述就是数据模式(data schema).
多级数据模式
外模式(external schema): 用户端所用到的部分
数据的描述
概念模式(conceptual schema): 用逻辑模型对一
个单位的数据的描述
内模式(internal schema): 物理数据模型对数据
的描述
数据独立性
逻辑独立性：外模式对概念模式的抗扰性
物理独立性：概念模式对内模式的抗扰性
通过映射（mapping）增强数据独立性，降低数
据库系统的使用与维护代价, 增加优化灵活性
SUM: 数据模式是数据库的具体描述, 即数据细节
设计. 内模式描述数据如何存储, 概念模式描述数
据的逻辑内容, 外模式即视图, 描述用户可以看到
的内容
数据库的生存周期（Life Cycle）
数据库系统的规划
数据库设计
数据库的建立
数据库的运维
数据库的扩充和重构
CHAPTER 2 数据模型
数据模式（结果）需要数据模型（手段）来描述 SUM: 外\概念模式是逻辑数据模型部分, 内模式是物理数据模型部分, 这都属于具
体实现. 视图部分是概念数据模型部分, 属于设计部分.
传统数据模型(逻辑数据模型)
类型
层次数据模型
树形结构表示实体间联系
缺点：只支持二元一对多联系，数据操作不便（树形结构，存储复杂）
网状数据模型
有向图表示
缺点：结构复杂，语言复杂
关系数据模型
以集合论中的关系为基础
BASIS
属性：事物的特征描述
域：属性的取值范围
关系
关系的型为模式（Schema）/内涵（Intension），
值为实例/外延（Extension）
关系的性质
关系满足第一范式1NF条件，即域中值为原子数
据（Atomic data）
行、列次序无关，任意两行不能全同（集合性质）
列是同质的（Homogeneous）, 类型一致
关系可以用来表示一个实体，也可以是实体间关系
键
候选键（candidate key），简称键：关系中的一属性or属性
组唯一地决定一元组（决定性条件），but其任何真子集不可
以（最小性条件），则称其为键
e.g. 在关系STUDENT中，学号是键，属性组（学
号，入学年份）不是键，因为它的真子集{学号}是
键，而这种属性组叫做超键（super key）
一般从候选键中选一个作为主键（primary key），
其他为候补键（alternate key）
如果主键由all属性构成，则称为全键（all key）
e.g. 关系SUPPLY（供应商，零件名，工程名）
候选键中包含了的属性，称为主属性（prime
attribute）；all候选键中都没有的属性，称为
非主属性（non-prime attribute）
外键（foreign key）：外键是一个关系（表）中
的属性，它引用另一个关系的键，用来表示两个
关系之间的联系。上述例子中，先修课程号为外键，引用本关系
COURSE的主键
SUM: 整个集合称为一个关系, 每一行是其中一个成员, 可以是一个实
体 or 实体间联系. 每列是一个属性, 域代表其取值范围.
候选键是一组属性(数量>=1), 满足决定性 & 最小性条件. 从中选出
一个作为主键, 剩下的就成了候补键. 被候选键包含的属性, 是主属
性. 外键是引用其他关系的候选键(常为主键).
约束
定义（模式定义时申明） & 检查
类型
域完整性约束(Domain Integrity
Constraints)：属性值在域中取，可为NULL
实体完整性约束（Entity）：每个关系有一个
PK，且唯一
引用完整性约束（Referential）：FK必须引用实
际存在的PK或候选键，否则取NULL
一般完整性约束 / 业务规则（Business
Rules）：特定领域业务规则所决定
SUM: 约束 取值∈域/PK存在且唯一/FK存在 or 取NULL
操作
关系代数操作（筛选型操作）
选择（Selection）
选择行
投影（Projection）
投影列
集合操作
交、并、差
允许条件: 列数相同、数据类型相同、语义对应
拼接型操作
笛卡尔积
连接（Join）
与笛卡尔积的区别：多了连接条件
两个表拼接成一个表时，会合并属性列，而重复的
列（符合等号连接条件的），自然连接就会删去其
一。e.g. PERSON中有name，CLASSMATE中也
有name，合并后，可以只保留一个。
案例
SUM: 选择行, 投影列, 交并差操作同结构表, 拼
接操作(笛卡尔积+条件=连接)
关系演算
概念数据模型
非传统数据模型/后关系模型
ER（Entity-Relationship data model）
一种概念数据模型，主要用于系统分析与设计阶段。
BASIS
实体（Entity）
实体集：同类事物的抽象
实体：实体集中的实例
实体键/主键
弱实体：依附于其他实体（所有者实体）存在，
其 实体键（部分键）不能单独作为PK
e.g.
弱实体的主键 = (宿主实体的主键) + (弱实体自己的部分键)
属性（Attribute）
原子属性
非原子属性
组合（元组）属性 e.g. 街道：（邮编，街道名）
多值（集合）属性 e.g. 同学：{Jerry, Lilly, Henry}
联系联系的语义约束（结构约束）
基数比（Cardinality ratio）
二元联系
两个实体之间的关系，如学生实体和课程实体之
间的选课关系，就是多对多
多元联系指多个实体间的关系，如A,B,C之间的关系
自联系/递归联系
参与度（Participation）
ER图
规则
矩形：强实体
双线矩形：弱实体
椭圆：属性（+下划线表主键属性）
双线椭圆：多值属性
菱形：联系
弧上的数字：基数比
双线弧：全体参与
单线弧：部分参与
属性上短垂线：实体键or实体键的一部分
SUM: ER模型是概念数据模型, 内容为实体 &
属性 & 关系, 关键在绘图
扩充ER模型
特殊化（Specialization）：实体集的子集
e.g. 学生集可分为研究生、本科生等子集
U表示特殊化
单线：部分特殊化
双线：全特殊化
d表示不相交特殊化
o表示重叠特殊化
ISA三角：连接父子实体
普遍化（Generalization）：几个有共性的实体
集概括成的更普遍化的实体集 e.g. 男人集、女人集等等，可以概括为人集
聚集（aggregation）
参与某个联系的全部实体组成一个新实体（称聚
集），其属性是所有这些实体的属性及这个联系
的属性之并。
范畴（category）
不同类型的实体组成新实体，称范畴。
e.g. 房屋所有权归属，可能是住户（人），也可
能是开发商（公司）等等
SUM: 扩展ER = ER + 面向对象思想. 特殊化是
实体集->子集. 普遍化是多实体抽象. 聚集是关系
合并. 范畴是多实体合并.
O-O（Object-Oriented）非常适合处理复杂数据
基于逻辑（Logic）的模型
评价（传统数据模型，对比文件系统）
向用户提供了统一的数据模型（文件系统每个应
用自己定义文件结构，互不兼容）
数据和程序之间具有相当程度的独立性（文件系
统文件格式变了，程序需重新写）
向用户提供了统一的数据库语言
在数据共享性、安全性、完整性及故障恢复等方
面提供了足够的保障
CHAPTER 3 关系数据库语言SQL
SQL特点
SQL 语言主要分为哪几类？
用户接口
操作: 数据定义\查询\更新(增删改)\控制(访问权限)
对应语言:
Data Definition Language (DDL)
Query Language (QL)
Data Management Language (DML)
Data Control Language (DCL)
语言特点
面向记录(靠指针,一次访问一个记录)->面向集合(靠属性
值,一次操作得到一个记录的集合)
嵌入式(embedded)和交互式(interactive)
过程性(procedural)->说明性(declarative)
定义语言 DDL
数据类型
常用：VARCHAR(n)
常用：INT, DECIMAL(p, s)
常用：DATE
基表模式创建
CREATE TABLE <表名>(
 <列名> <数据类型>[<列级完整性约束条件>],
 <列名> <数据类型>[<列级完整性约束条件>] ,
 [<表级完整性约束条件>]
);
常见列级完整性约束条件
表级, 典例均可组合多列属性
e.g.
基表模式的修改与撤销
增加列 ALTER TABLE <表名> [ADD <新列名> <数据类型> [完整性约束]];
CREATE TABLE Student (
 s_id VARCHAR(20),
 s_name VARCHAR(50)
);
ALTER TABLE Student ADD enrollment_date DATE NOT NULL;
删除基表 DROP TABLE <表名>; DROP TABLE Temp_Test;
删除属性列(间接)
将保留的列复制到另一个新表, 删除原表, 重命名
新表为原表名
-- 第一步：创建一个新表，只包含你想要保留的列
CREATE TABLE Student_New AS
SELECT s_id, s_name, enrollment_date
FROM Student;
-- 第二步：删除原来的旧表
DROP TABLE Student;
-- 第三步：将新表重命名为原表名
ALTER TABLE Student_New RENAME TO Student;
修改属性列 ALTER TABLE <表名> MODIFY <列名> <数据类型>; ALTER TABLE Student MODIFY s_name VARCHAR(100);
主键
补充定义主键 ALTER TABLE <表名> ADD PRIMARY KEY(<列名表>); ALTER TABLE Student ADD PRIMARY KEY(s_id);
撤销主键定义
ALTER TABLE <表名> DROP PRIMARY KEY;
-- 暂时删除主键
ALTER TABLE Student DROP PRIMARY KEY;
-- ... (在此处执行批量插入数据的操作) ...
-- 插入完成后，记得加回来！
ALTER TABLE Student ADD PRIMARY KEY(s_id);
暂时撤销主键定义，在插入新的元组时，可以提
高系统的性能
外键
补充定义外键
ALTER TABLE Student
ADD CONSTRAINT fk_student_class
FOREIGN KEY (c_id)
REFERENCES Class(c_id);
撤销外键定义 ALTER TABLE <表名1> DROP <外键名>; ALTER TABLE Student DROP CONSTRAINT fk_student_class;
其他模式对象的定义与撤消
定义与撤销别名
-- 1. 定义别名 (CREATE SYNONYM)
-- 需求：嫌 Student 这个名字太长，想用 Stu 代替
CREATE SYNONYM Stu FOR Student;
-- 2. 撤销别名 (DROP SYNONYM)
DROP SYNONYM Stu;
索引的建立与撤销
次序: ASC(默认升序)\DESC
UNIQUE表明each索引值对应唯一数据记录
已含重复值的属性列不能加UNIQUE索引
加了后, 相当于加了UNIQUE约束
CLUSTER表示要建立的索引是聚簇索引
CREATE INDEX idx_enroll_date ON Student(enrollment_date);
DROP INDEX idx_enroll_date;
举例
查询语言 QL
SELECT 需指明要查询的项目（列名 or 表达
式），* 表示 all
FROM 指明被查询的表 or 视图名， 可取别名
（本句有效）
WHERE 说明查询条件，加 DISTINCT(作用于
all 目标列) 去重 WHERE job=’salesman’;
WHERE sal>5000;
WHERE comm IS NULL;
WHERE sal BETWEEN 3000 AND 5000;
如果null参与算术运算，值为null
如果null参与比较运算，则结果可视为false。在
SQL－92中视为unknown。
如果null参与聚集运算，仅 count(*)考虑null
字符匹配查询
%代表任意长度（可为0）的字符串
_ 代表任意单个字符
\转义符, \% 和 _
属于判断查询
连接查询
多表连接
单表连接: 查询薪水超过其部门经理的职员、及
其经理姓名
存在查询
存在返回 True， 否则返回 False
子（嵌套）查询
ALL 是谓词，返回布尔值
AVG是函数，返回计算结果数值
分类
不相关子查询：子查询独立于外部查询，只执行
一次，结果供外部查询使用。
相关子查询：子查询的执行依赖于外部查询的变
量（如引用了外层的表别名）。外部查询每处理
一行，子查询都要重新执行一次。
查询结果分组
对比
WHERE 子句和 HAVING 子句有什么区别？
查询结果排序
2为列序号, 指代第二列, 即job
deptno默认ASC, job DESC
集合操作
并操作 UNION 要求并兼容(属性数同, 对应域也同)
交操作 INTERSECT
差操作 MINUS
例题
1. NULL
2. False（Unknown）
【除法运算/双重否定】
查询选修了全部课程的学生姓名。
SELECT Sname
FROM Student
WHERE NOT EXISTS (
SELECT * FROM Course
WHERE NOT EXISTS (
SELECT * FROM SC
WHERE SC.Sno = Student.Sno
AND SC.Cno = Course.Cno
));
--- 语义：不存在一门课程，该学生没有选修。
操纵语言 DML
插入
单行直接插入
多行间接插入
修改
删除
视图
概念
是由其他表（基表/视图）导出的虚表
作用
简化用户操作\多视角看数据\安全保护数据\逻辑
独立性
定义 & 撤销视图
CREATE VIEW <视图名>(属性列表)
 AS <选择条件>
DBMS执行CREATE VIEW语句时只是把视图的定
义存入数据字典，并不执行。后续需消解查询
DROP VIEW <视图名>;
删除DD中 指定视图定义
由该视图导出的其他视图须显式删除
删除基表时, 其到处的视图须显式删除
查询数据
to用户, 同基表查询
to系统, 通过消解(DBMS完成): 从数据字典中取出
视图定义 -> 转换为基表查询 -> 执行基表查询
更新数据
严格规定：含主键的单表行列子集视图可以更新
举例
嵌入式SQL
CHAPTER 4 DBMS引论
解释执行的RDBMS结构
事务(Transaction)
事务是DBMS最小执行单位, 是对数据库的操作
序列, 操作 顺序-串行执行
定义：用户定义的一个DB操作序列，要么全做，
要么全不做，是一个不可分割的工作单位。
ACID性质
原子性Atomicity: 操作不可分割
一致性Consistency: 使DB从一致性状态到另一
个一致性操作
隔离性Isolation: 多事务执行互不影响
持久性Durability: 执行影响持久保留
结束方式
提交Commit: 操作全部执行成功
回滚Rollback: 操作全部不执行
可显式\隐式
DBMS进程结构
进程: 独立程序代码的一次动态执行
举例解释
e.g.
× 微信是一个进程, 微信里的每一个聊天框是一个线程
√ 在一个设备上，微信整体大致是一个主进程（App）+ 多个线程。主线程负责
UI 显示，其他线程负责后台任务。如媒体线程\IO线程\网络线程\DB线程。
一条朋友圈内容或点赞动作只是一个数据项或一个事件
你点了 5 次赞，不是开了 5 个线程；
而是任务队列中排入 5 个“HTTP 请求任务”，
由网络线程池（比如 4 个工作线程）轮流处理。
分类
应用进程（用户进程）
DBMS进程：对应DBMS代码的一次动态执行
核心进程（服务器进程）: 直接处理用户请求
(SQL查询、事务等) 补充（服务器区分）
硬件层：通常理解的服务器，是提供计算、存储等硬
件资源的一台物理计算机 or 划分出来的虚拟机。如
数据库服务器、Web服务器、邮件服务器
操作系统/网络服务层: 持续在后台运行, 等待并
响应网络上其他客户端请求的一个软件程序. 如
Apache服务\MySQL服务器程序.
DBMS内部层: 特指在数据库管理系统 (DBMS)
内部，为了处理每一个具体的用户连接和请求而
创建的独立工作进程或线程。
云计算层: 物理服务器集群中划分的计算资源包,
本质是一台虚拟机.
后台进程: 维护数据库系统内部的运行
补充(动态)
动态: 在程序运行时发生的
静态: 在程序编译或准备阶段就已经确定好的
Oracle实例 = DBMS进程 + 系统全局区(SGA)
SGA 是一个共享的内存区域，包含数据库实例的数
据和控制信息。所有服务器进程都可以读取 SGA。
SMON (System Monitor) 进程
DBWR (Database Writer) 和 LGWR (Log
Writer) 的区别与联系?
PGA (Program Global Area) 与 SGA 的主要区
别是什么？
进程结构方案
单进程结构应用代码和DBMS代码结合成单个进程而执行。
e.g. MS-DOS上的单用户Oracle。
多进程结构:
- 1个用户应用对应1个用户进程
- 使用多个进程/线程执行DBMS
应用代码与DBMS代码在同一个进程（称用户进
程）中运行，程序接口维护隔离及传送数据。
(User/Server相结合的进程结构, 单任务)
<嵌入式架构>DBMS代码在用户应用进程内部运
行, 仅单个进程
为每个应用进程建立一个DBMS核心进程，称专
用服务器进程(两任务: client+server)
<客户端/服务器架构>DBMS代码在一个独立
的、专门的服务器进程中运行, 至少两个进程
(当下主流)单核心进程\多线程DBMS(共享服务器进程)
不使用OS提供的多线程机制，而由DBMS自己实现多线
程机制
DBMS系统结构 (DBS体系结构)
集中式DB
运行于分时系统环境(即主机/终端系统)
“头脑只有一个，手脚有很多”
优点: 易于管理\控制\维护
缺点: 终端用户过多会导致host任务繁重, 性能下
降; 系统可靠性依赖主机
运行于PC或PC LAN环境 (文件服务器架构)
“胖客户端，傻服务器”
网络传输量巨大，数据安全性差，且容易导致数
据不一致（并发控制难）。
运行于Client/Server环境 (两层) 结果：大大减少了网络流量，减轻了客户端负
担。
运行于Client/Server环境 (三层)
集中式数据库系统的缺点
通信开销大
性能差：容易出现单点失效问题
可用性差
可扩充性差
分布式DB
物理上分布、逻辑上集中
物理上分布、逻辑上分布
数据字典DD
即元数据, 包含DB的定义信息\描述信息\统计信
息\约束信息等管理信息
结构
核心: 一组加密的基表 (Base Tables)，由系统管
理，用户严禁直接访问。
接口: 一组只读视图 (Views)，建立在基表之上，
供用户安全查询。
Oracle中的DD视图分类
静态视图 (Static Views) - 权限三层级:
DBA_*: 上帝视角，看数据库里的一切。 (DBA专
用)
ALL_*: 朋友视角，看自己的东西 + 别人授权给
你看的东西。
USER_*: 自己视角，只看自己的东西。 (普通用
户最常用)
USER_ ⊆ ALL_ ⊆ DBA_
动态视图 (Dynamic Views) - 实时监控:
以 V$ 开头 (如 V$SESSION, V$PROCESS)。
数据来自内存，反映数据库当前的活动状态。
用于性能监控和故障诊断。
CHAPTER 5 DB存储结构
DB存储结构
多级存储
二级存储: 内存 - 外存
三级存储: 主存 - 磁盘存储器 - 辅助存储器
存取时间开销:
- 活动头的移动时间－寻道时间
- 磁盘片的旋转定位时间－等待时间
- 读/写数据时间－传输时间
主存与磁盘存取速度不匹配, +缓冲区, 并采用数据预取
(Prefetching)和延迟写(Delayed Writing)技术减少IO
数据库会遭破坏, 提供数据后备(Backuping)
物理结构
DB以多个文件形式存储, 物理存储于硬盘
数据文件: 数据 & 元数据
日志文件
控制文件: 保存与DB相关的若干参数, 如DB名, 日
志文件名及地址, DB建立日期等
逻辑结构
逻辑存储空间
表空间: 一个DB可包含一个or多个
段(Segment): 属于一个数据库对象（表、索引
等）的全部空间。
范围(Extent): 一个段由一组范围组成, 是空间分
配的逻辑单位
数据块: 一个范围由一组连续的数据块组成, 最小
物理单位
用户模式及其对象
每个数据库用户对应一个模式(Schema， 命名空间)。
模型对象是一个模式中包含的数据逻辑结构对象. 如 表
\视图\索引等
关系DB中表的典型存储机制
索引
索引是一种独立于数据表的、排好序的数据结构。
它存储了表中一个或多个列（索引键）的值，以及一个指向该值对应的数
据行在磁盘上物理位置的指针。
常采用B+树(自平衡多路查找树)
元组按照主键值有序排列，可采用二分查找法
索引键
如果是PK或UNIQUE列, 称主索引(唯一索引, 由
DBMS自动建立), 否则是次索引(自定义).
如果是列组, 称组合索引
若each索引键值有对应索引项, 称稠密索引, 否则
非稠密索引
在顺序文件上的索引
稠密索引
速度>直接在数据文件上查找
一般, 一个物理块中可以存放的索引项的个数>>记录数。IO次数少
索引项有序排列, 可二分查找; 数据文件无序
索引文件小, 可在内存操作, 而不必访问磁盘
e.g.
非稠密索引
二分查找 索引文件中索引键值<=K，且最接近K的索引项
根据找到的索引项中的记录指针到数据文件中读取相应的物理块 D, 并
二分查找主键值为K的记录。
稠密索引中的每个索引项对应数据文件中的一条记录，
而非稠密索引对应一个物理块。
只能用于顺序文件上的索引组织
稠密索引与稀疏索引的区别
多级索引
非顺序文件中的索引结构
if 索引键值唯一, 可建一级稠密索引, 再在此基础
上建立多级非稠密索引
else if 不唯一, 在第一级的稠密索引和数据文件
之间加一个记录指针桶(bucket), 桶内存放着索引
键值为Ki的记录的记录指针
B树
在B/B+树中，由叶结点所构成的最下面的一级索引总采用稠密索
引，而其它层次上的索引则采用非稠密索引。
结点
每个结点占用一个物理块，每个结点能容纳 n 个
键和 n+1 个指针（指向下面的结点）
例如：设每个物理块的大小是4096个字节，每个
键值占4个字节，每个指针占8个字节，则满足：
4n + 8(n+1) ≤ 4096的最大 n 的值是340。
B树规则
秩=n
关键字数要求
叶子结点
根节点
内部节点
例如：有一棵秩n=3的B+树，则每个结点最多可
放3个关键字和4个指针，每个叶子结点至少有2
个关键字和3个指针，每个内部结点至少有1个关
键字和2个指针。
非叶节点
优点：表较大时，索引可提高查询性能，特别是
适合指定行&范围查询
缺点：增加系统开销，特别是数据频繁更新时
散列
散列是通过一个散列函数（Hash Function），将索引键的值直接计算
（映射）成一个存储地址（或桶 Bucket 的地址）。
多使用动态散列（桶的数目可以分动态变化；桶
可分裂/合并）
簇集
簇集不是一个独立的数据结构，而是一种物理存储策略。它规定了数据行在磁
盘上的物理排序方式，是根据一个或多个列（簇集键, Cluster Key）的值来组
织的。数据本身就是排好序的。
当插入一行新数据时，数据库会根据其簇集键的值，找到磁盘上对应的物理页
面（Data Block），并尝试将其与其他具有相似键值的数据行存放在一起。
核心作用：强制将逻辑上相关的数据，在物理磁
盘上也存放在一起。
使用 CLUSTER 指令牺牲写入性能，换取极致的
读取性能。
实现
索引簇集: 对簇表在簇集键上再建索引，每个簇
集键值有一个索引项。
散列簇集: 对簇表的行在簇集键列上运用Hash函
数进行散列，以决定相应数据块的物理地址。
总结
索引: 有序索引(稠密\稀疏), B+树索引, 多级索引
散列: 静态散列(固定桶), 动态散列(桶数可变), ROWID直接存取
簇集: 多表簇集(相关表存一起), 表内簇集
CHAPTER 6 查询处理和优化
BASIS
查询处理: 从DBMS接受一个查询请求到返回查
询结果的处理过程
查询处理（Query Processing）的四个主要步骤
查询优化(Query Optimization)
目的: 提高性能\方便用户(用户不必考虑效率问题,
由系统优化)
两个层次
代数层：查询所对应的关系代数表达式->抽象的
语法树，等价变换，减小中间结果大小
物理层：选择高效的存取路径，提高磁盘I/O效率
两种策略
基于规则（Rule）：运用启发式规则确定理论
“最优”执行计划
基于代价(Cost)估算：先确定几个理论“较优”
执行计划，后根据DD中的参数估算执行代价(时
间开销)，选择代价最小者
代数优化
目标: 对查询进行等效变换，尽量减小查询中间
结果的大小
启发式规则:
先选择\投影, 后连接\集合
连接先连小关系间
笛卡尔积+选择->连接
对原始关系加必要投影
查询树（Query Tree）
定义：查询树是 SQL 语句的关系代数表示形
式，树叶是关系，非叶节点是操作。
优化体现：通过等价变换规则（如交换律、结合
律），改变树的结构（如将选择节点下移到叶子附
近），从而生成一棵等价但执行代价更小的树。
e.g.
优化策略
依赖于存取路径的规则优化：优化器根据底层物
理存取方式来决定查询执行计划。
独立于存取路径的优化(代数优化)：只在逻辑层面对
查询进行代数等价变换，不考虑底层访问机制。
代价估算优化
一个操作的代价
一个查询执行计划的代价
只需比较不同计划的I/O次数 or 数据块数
例子:
嵌套循环连接(遍历)
利用索引(或散列)寻找匹配元组法
(ns / NB): 假设连接键的值是均匀分布的，S中总
共有 ns 个元组，连接键有 NB 个不同的值，那
么平均每个值对应 ns / NB 个元组。
排序-合并连接（Sort-Merge Join）的优缺点
CHAPTER 7 事务管理
事务管理: 保证事务始终满足ACID准则.
数据库恢复: 系统故障时
并发控制: 并发执行时
数据库恢复
恢复的基本技术
三类故障
事务失效: 发生在事务提交完成前. 如事务运行出
错\用户撤销\系统中止
系统失效: 掉电 or 系统故障, 内存数据丢失, 外村
数据库未遭破坏
介质失效
三种恢复技术
基本原理：利用冗余数据重建数据库
数据后备复本(Backup)
周期性把磁盘上的数据库物理文件复制到磁带上
后备时，需暂停DB以保证一致性
-> 数据量大, 后备费时
-> 数据恢复时, 只能恢复至最近的后备点
大型DB不用, 继承自文件系统
运行记录、日志(Log)
根据Log去撤销与重做
如果DB受损, 则先回到最近的Backup再将所有已
经提交的事务重做
日志包含:
事务及其状态
前像&后像: 即更新前后的数值
多复本
独立失效模式: 在不同的、物理上独立的 系统上
保存多份完全相同的数据库拷贝（复本）, 所有
数据不会一起失效.
日志结构与机制
回滚段文件
重做日志文件
日志内容
活动事务表 ATL: 执行中未提交的事务标识 TID
提交事务表 CTL
前像文件: 记录旧值 BI, 用于Undo操作
后像文件: 记录新值 AI, 用于Redo操作
一组文件, 循环写入日志(写满一个写下一个), 称
在线日志, 写满就复制到归档日志(磁带上)
更新事务的执行与恢复
规则
先记后写
写入DB文件前, 先将操作(包含BI)记入日志
为了进行 undo, 防止系统故障使得先写入的数成
为脏数据(保障过程)
提交规则
commit前, 将AI\Commit标记 写入日志
为数据丢失后 redo(保障结果)
解释写数据: 磁盘->内存, 先在log上记录BI, 写入内
存, 再在log上记录AI, commit告知用户成功
了. 空闲时刻, 再从内存写回磁盘.
三种方案
<1> TID→ATL
<2> BI→Log /* 先记后写规则 */
<3> AI→DB /* 提交规则 */ /* AI直接写入数据库 */
<4> TID→CTL
<5> 从ATL删除TID
<1> TID→ATL
<2> AI→Log /* 提交规则 */ /* AI先写入日志文件 */
<3> TID→CTL
<4> AI→DB /* AI再写入数据库 */
<5> 从ATL删除TID
<1> TID→ATL
<2> AI、BI→Log /* 满足两条规则 */ /* AI先写入日志
*/
<3> AI→DB /* AI部分写入DB */
<4> TID→CTL
<5> AI→DB /* AI全部写入DB/
<6> 从ATL删除TID
失效的恢复对策
实例恢复
事务失败: 先undo, 再从ATL删该TID, 释放资源
(系统自动)
系统失败: 重启OS和DBMS (DBA执行), 利用日
志undo & redo(系统自动)
介质恢复
修复系统, 重启OS & DBMS.(系统管理员 or
DBA)
加载最近backup (DBA)
利用AI redo(系统自动)
Backup (备份)：用于防灾。解决的是介质故障。
什么是 Checkpoint (检查点)？它的作用是什么？
并发控制
BASIS
数据库系统中，事务必然是并发访问的。否则性
能降低(访问时间长)\资源利用率低
并发引起的不一致问题
丢失更新: 写写冲突
读脏数据: 写后读
读值不可复现: 读后写
情况1: 值修改
情况2: 增删了记录, 也叫幻读
正确性准则
多事务并发执行, 既要交叉执行以充分利用系统
资源, 又要避免访问冲突
调度
串行调度: 事务串行执行
并行/并发调度: 事务交错执行
可串行化调度
一个并发调度，虽然指令是交错执行的，但其最终
结果与某一个串行调度的结果等价。
(可串行化是并行事务正确性的唯一准则) 为什么说"某一"串行调度?
DB的并发控制 不关心执行的确定性(结果是什么), 只
保证数据的一致性(等价于任意一个串行调度)
e.g. 两人抢票, 不关心谁抢到, 只关心结果正确
两种等价
视图/目标等价: 两调度, 在任意初态, 若从DB中读
出的数据一致, 终态也一致, 则两调度等价
冲突等价: 通过调换不冲突操作次序而得到的新
调度。
注: 目标等价更普遍
e.g.
1. 冲突分析（找不同事务对同一数据的冲突操作）：
- C 数据：w1(C) 先于 r3(C) -> 边 T1 -> T3
- A 数据：r1(A) 先于 w3(A) -> 边 T1 -> T3
- B 数据：r2(B) 先于 w2(B) (同事务，非冲突)
- 其他无冲突。
2. 优先图：
- 节点：T1, T2, T3
- 边：T1 -> T3
3. 判断：
- 图中没有环（Cycle）。
- 结论：是冲突可串行化的。
4. 等价串行顺序：拓扑排序结果，如 T1 -> T2 -> T3 或 T2 -> T1 -> T3。
冲突可串行化 -> 目标可串行化
但由于检测算法是NP完全问题, 目标可串行化不适用(判断是否目标可串行化, 需要和
all 可能的串行调度比较, 计算量随事务数量爆炸)
而冲突可串行化可通过简单算法判断, so 以其为具体的正确性准则
(虽然会漏掉一些可串行化调度)
NP完全问题, (Non-deterministic Polynomial) , 多项式复杂程度
的非确定性问题
由于并发访问量大, "检测"思路效率过低, 采用"协
议"思路, 即采用加锁协议
加锁协议
X锁（eXclusive lock）：排他独占
任何事务的任何操作(R/W)前, 须获加X锁, 完成
后释放. 每个X锁必须保持到事务结束(End of
Transaction, EOT).
保证正确性和一致性, 但降低并行性
合式事务: 遵循先加锁, 后操作的事务
(S，X)锁
X锁：用于写操作。
S锁：(Sharing)用于读操作。
S锁使得可以并行读, 阻止写
X锁使得独占写
活锁现象: 数据对象始终被S锁占有
预防: FCFS, 先来的X锁先进入等待队列, 阻塞后
面的S锁进入
(S, U, X)锁
U锁：共享更新锁(Sharing Update Lock), 让写
者可以先拥有更新的权力, 等最后真写入时, 再升
级成X锁. 期间可加S锁
两阶段封锁协议 (2PL)
合法: 遵守基本加锁规则, 即一个事务在访问数据
前加锁（L），在访问结束后释放锁（U）
不可串行化： 指这个调度的最终执行结果与任何
可能的串行执行的结果都不相同。
2PL规则: 一个事务分为两个阶段：
- 增长阶段 (Growing Phase)： 只能获取锁 (L)，不能释放锁 (U)。
- 缩减阶段 (Shrinking Phase)： 只能释放锁 (U)，不能获取新锁 (L)。事务一旦释放了
第一个锁 (U)，就进入此阶段。在此阶段，事务只能释放锁，不能再获取任何新锁。
针对的是同一个事务, 该事务的所有L在一起, 所
有U在一起, 靠锁点分隔
遵守2PL是保证事务任何调度可串行化的充分不必要条件
2PL 协议为每个事务定义了一个“锁点” (Lock
Point), 即事务第一次释放锁 (U) 的时间点
两冲突事务因为是对同一资源的冲突, 所以执行有先后
(先进去的加锁), LP有先后, 不会成环, 所以可串行化
加锁由DBMS中的加锁管理器同意我管理, 有一
张缩表, 记录锁的事务持有和申请情况(事务, 锁,
数据对象)
Oracle中，锁的获得与释放
获得
X锁
显式： LOCK TABLE 命令
隐式： INSERT, UPDATE, DELETE
S锁 LOCK
SU锁
显式：LOCK
隐式：SELECT ... FOR UPDATE命令
释放
COMMIT/ROLLBACK(事务结束) 2PL 中“释放阶段”的标准操作
DDL(Data Definition Language)操作
执行 DDL 命令时，会强制执行一个“自动提
交” (Implicit COMMIT)。
程序终止
应用程序异常崩溃，数据库会强制对这个会话所
做的一切执行自动 ROLLBACK。
LOGOFF（终止会话）
如果你的会话（Session）在退出时，还有一个
未提交的事务，数据库的标准行为是自动替你执
行 ROLLBACK。
多粒度封锁与意向锁
封锁粒度: 一把锁可以封锁的数据对象大小(逻辑\物理数据单元)
单粒度封锁: 固定一种粒度
多粒度封锁
选择合适的粒度: 大粒度可以减少锁的数量, 小粒
度可以减少锁内冲突, 增加并发性
多粒度树
对任一节点可以独立加锁（显式），其all子节点
也加同样的锁（隐式）
so对一个节点加锁时， 要注意：
该节点锁冲突
子节点锁冲突
祖节点锁冲突
检查十分复杂。比如要对关系R加锁，要检查所
有R下元组
意向锁
对一个节点加锁，先对其上层节点加意向锁
故申请锁时， 从上至下申请
如R下有元组加锁，在R上就要加意向锁进行告
知，R上加真锁时， 就不用一个个检查子节点了
意向锁的作用?
category
IS锁（意向共享锁，Intent Sharing Lock）：某
些子节点拟加or已加S锁
IX锁： 子节点X锁
含义：“我不锁这张表，我只是打个招呼，我打
算去下面的某一行加X锁。”
在表级别，两IX可兼容，分别去第1行和第2行加X锁，互不干
扰。这就是意向锁实现“行级并发”的关键。
SIX锁：本身S锁，子节点X锁
S & X被同一事物持有， 所以不会矛盾
用于扫描全表，并更新其中一部分的场景
该表定义不同锁是否可在同一个数据对象上共存
目的： 允许并发 & 提供保护
e.g.
SELECT * FROM Movie WHERE title = 'A';
UPDATE Movie SET year=1999 WHERE title = 'B';
Movie 加IS锁，对A加S锁
Movie 再加IX锁，对B加X锁
死锁的检测\处理\预防
e.g. R1(A);R1(B);R2(B);W1(B);R2(A);W2(A); 上了S锁，无法上X锁
因为2PL协议，一事务无法在获得X锁前，释放S锁
预防法
顺序申请
一次申请
选择性地rollback & retry事物
利用时间戳应付锁冲突
等待死亡策略
站在新来的事务T1的角度，若T1更老，则等待T2
释放锁；若T1更年轻，则自杀
老则等，幼则死
击伤等待策略
若T1更老， 击伤T2（强制终止）；若T1更年
轻，等待T2释放锁
幼则等，老则抢
共性: 牺牲年轻事务
OS中思路，不适用DB
解除法
超时死锁检测法
事务执行时间超时
锁申请的等待时间超时
等待图法周期性检测等待图中有无回路
处理选择一个事物撤销
CHAPTER 8 DB的安全与完整性约束
安全性是防止‘非法用户’破坏数据（防坏人）；
完整性是防止‘合法用户’输入不符合语义的数据（防错数）。
DBMS的安全机制
用户标识与鉴别 (Authentication)：通过用户名、密码、数字证书等方式，验证用户合法性。
存取控制 (Authorization) DCL：通过 GRANT（授权）和
REVOKE（撤销）命令，定义用户权限。
e.g.
1. UserA 授权：
`GRANT SELECT ON Employee TO UserB WITH GRANT OPTION;`
2. UserB 授权：
`GRANT SELECT ON Employee TO UserC;`
3. UserA 收回：
`REVOKE SELECT ON Employee FROM UserB CASCADE;`
4. 结果分析：
UserC 的权限会被收回。
原因：标准 SQL 的 REVOKE 默认或强制（CASCADE）会收回源于该用户
的所有权限链。UserB 的权限没了，他授给 UserC 的也就失效了。
1. 创建角色：
CREATE ROLE Staff_Role;
CREATE ROLE Manager_Role;
2. 员工授权：
GRANT SELECT ON SalaryTable TO Staff_Role;
GRANT UPDATE(Phone) ON EmployeeTable TO Staff_Role;
3. 经理授权：
-- 角色继承：经理拥有员工权限
GRANT Staff_Role TO Manager_Role;
-- 经理特权
GRANT UPDATE(Amount) ON SalaryTable TO Manager_Role;
4. 分配给用户：
GRANT Staff_Role TO User_Alice;
GRANT Manager_Role TO User_Bob;
视图机制 (View)：
视图是一个“虚拟表”，是基表的子集。通过对“视图”授权，
可以在逻辑上隐藏特定的行\列
审计 (Audit)：
记录特定用户或特定操作的所有活动日志。当安
全事件发生时，DBA可根据日志溯源语句\特权\模式对象审计
数据加密 (Data Encryption)
静态加密 (At Rest)： 保护存储在磁盘上的数据文件。
动态加密 (In Transit)： 保护在网络中传输的数据（如SSL）。
完整性约束
规则组成
设置
检查
处理
类型
静态约束: 针对DB的静态状态
(某个时刻的数据)
固有约束(数据模型自身固有), 如关系型数据库中
的1NF条件
隐含条件, 隐含在数据模式中, 通常须DDL声明,
分为域\实体\引用完整性约束
显式约束: 语义层面更广的约束，通常与特定应
用领域有关, 如工资不能为负等
动态约束: 针对DB的动态变化(如各种操作), 同样
与特定应用领域有关，如账户余额减少后不得<0
SQL实现
在基表定义中
CREATE TABLE dept (
 deptno INT PRIMARY KEY CONSTRAINT pk-dept,
 dname VARCHAR(10) UNIQUE CONSTRAINT unq-dname,
 loc VARCHAR(10) CHECK(loc IN('China', 'England', 'America')) CONSTRAINT ck-loc
); pk-dept 为约束名
断言（全局约束）
CREATE ASSERTION <name> CHECK(<condition>);
DROP ASSERTION <assertion-name-list>;
[例]: (对单个关系的整体)每门课程的选修人数不
少于20人。
CREATE ASSERTION ass_1 CHECK (
20 >= ALL (SELECT COUNT(*)
FROM SC
GROUP BY CNO)
);
[例]: (对多个关系间的联系)学生在选修‘数据结构’(DS)课程之前
必需先选修过‘C++程序设计’(CPP)课。
CREATE ASSERTION ass_2 CHECK (
 NOT EXISTS (
SELECT * FROM SC
WHERE CNO IN (SELECT CNO FROM Course
 WHERE CNAME = ‘DS’)
AND
SNO NOT IN (SELECT SC1.SNO
 FROM SC SC1, Course
 WHERE SC1.CNO = Course.CNO
 AND Course.CNAME=‘CPP’)
));
CHAPTER 9 触发器与主动DBS
主动DBS
D: 能根据发生的事件, 主动进行一些操作(功能科
事先定义)
规则机制
条件 - 动作规则 CA Rule: 条件满足, 执行动作
事件 - 条件 - 动作规则 ECA Rule: 事件发生,
DBMS检测条件, 执行动作
触发器与声明式完整性约束相比，有什么优缺点？
ECA规则
事件: SQL操作语句
动作
执行频度
EACH ROW: 行被操纵, 动作执行一次
EACH STATEMENT: 一个事件， 动作执行一次
对比
执行时刻\方式
INSTEAD OF：动作替代事件而执行
分类
SQL表示
执行方式
结合方式
动作操纵语句作为事件操纵语句所在用户事务的
一部分被执行
立即执行： 一旦触发事件， 动作即作为事务一
部分而被执行
推迟执行: 动作操纵语句推迟到事务末尾 EOT 执
行（but 实现复杂， 很少有DBMS实现）
分离方式动作操纵语句是衍生事务， 在事务提交后提交
连锁触发的对策： 正确控制ECA的嵌套执行，防
止循环触发而无休止执行。一般为触发次数规定
一个上限， 达到后DBMS强制撤销 all
实现
松耦合：在应用层和DBMS间，加主动DB功能模
块，用于捕获触发事件、检测条件、向DBMS提
交动作
优点： 无需修改传统DBMS
缺点：通信开销大、性能差
紧耦合：主动DB功能集成到DBMS中
嵌入法：二者折中（只能处理简单规则）
(E) 事件检测： DBMS 感知到事件，立即向外部
模块通知
(C) 条件评估： 外部模块被“唤醒”，在外部评
估条件。
(A) 动作执行： 外部模块向 DBMS 提交一个新
事务来执行动作。
触发器应用
完整性约束的维护
导出数据实时更新
数据库多副本一致性的维护
触发器的外部应用
将特定应用领域的业务规则，抽象成ECA规则
CHAPTER 10 数据依赖与关系模式规范化
BASIS
关系模式的设计
bad 关系模式
方案1: R (SNO, CNO, G, T, DEPT)
方案2:
R1 (SNO, CNO, G)
R2 (CNO, T)
R3 (T, DEPT)
比较角度
数据冗余度方案1中T & DEPT重复多次
增删改操作
方案1 中
张乐若调到土木系, 只改其中一个元组值, 则数据不一致
若C01课取消, 把前三行都删了, 则关于张乐的信息就随之被被删除了
若新增课程, 尚无人选修, 则无法构造元组存储
good 关系模式规范化: 语义单纯化 -> 模式分解
函数依赖 FD
分类：完全/部分FD，平凡/非平凡FD，直接/传递FD
D: 一部分属性Y依赖于另一部分属性X的取值，
存在数据依赖关系 X → A，意思是 "X的值唯一确定A的值"
分裂、合并规则
分类
平凡/非平凡
平凡依赖
A, B为某模式两属性集， A -> B
B为A的子集 (Sno, Cno) -> Sno
平凡依赖无实际意义
非平凡依赖 B-A非空 (Sno, Cno) -> (Sno, Grade)
完全非平凡依赖 A与B交集为空 (Sno, Cno) -> Grade
部分/完全
完全依赖
部分依赖
直接/传递传递依赖
范式
1NF: 表中的每一列（属性）都是不可分割的原子值
2NF: 以1NF为基础，表中所有非主键属性，都必须
完全依赖于整个主键，而不是“部分依赖”反例, 主键为A & B, but C指依赖A, 不依赖B
3NF: 以1NF为基础, R的任一非平凡函数依赖X->A
满足下列两个条件之一: (1) X是超键, (2)A是主属性
BCNF范式：以1NF为基础, R的任一非平凡函数
依赖X->A满足：决定子X必是超键
即，在一个关系模式R中，只要存在一个有意义
的函数依赖（非平凡FD），那么这个依赖的"决
定因素"（决定子X）必须是一个超键。
2NF 的目标是消除部分依赖（非主属性对候选键的部分依赖）。
3NF 的目标是消除传递依赖（非主属性对候选键的传递依赖）。
BCNF 必然是 3NF，自然也就消除了部分依赖和传递依赖
Question？
为什么要消除部分依赖和传递依赖？
core：会导致数据冗余，进一步导致更新异常
只要有冗余，就必然伴随着
插入、删除、修改异常。
部分依赖案例
学生选课表 R(学号, 课程号, 成绩, 学生姓名)
{学号} → 学生姓名 是部分依赖
数据冗余：如果某学生选多门课，学生姓名就会重复存储多次
插入异常：无法插入未选课的学生
删除异常：删掉一门课，学生信息也随之被删
修改异常：如果学生改名，就需要修改他选的所有课程，漏了一条就会导致数据不一致
传递依赖案例
学生表 R(学号, 学生姓名, 学生系别, 系主任)
{学号} → 学生系别 → 系主任 是传递依赖
数据冗余：100个学生，就要写100次系主任名（应该放在系部表）
插入异常：设立一个新系，但是还没招收学生，就无法录入系主任名
删除异常：一个系就一个学生，毕业了，系主任就随之消失
修改异常：换了新系主任，需要修改全部记录
二者可以通过开一个新表存放解决
BCNF 是如何解决数据冗余和更新异常的？
存在一个决定子 X，它不是超键
因为 X 不是超键，所以 X 的值允许在表中重复出现。
因为存在函数依赖 X → A，所以每当 X 重复时，A 的值也必须
跟着重复
结果： (X, A) 这对"事实"被重复存储了。这就是数据冗余。
全键关系 因为没有非平凡FD，所以必然∈BCNF
两属性关系 必然∈BCNF
无损分解（细胞分裂法）
设有关系模式R（XAB），其中X，A，B均是R的属性（组）
非平凡依赖X->A, X不是超键，违反BCNF，进行分裂
R1∈BCNF，但有可能R2∉BCNF，可继续分裂R2
举例
关系 M (title, year, length, color, star, star-gender, star-add, studio, studio-add, studio-class)
存在以下FD：
(1) star ->star-gender, star-add
(2) studio -> studio-add, studio-class
(3) title, year -> length, color, studio
故Key = {title, year, star}。违反BCNF
模式分解
针对式（1）分解
STAR (star, star-gender, star-add) ∈BCNF
M1 (title, year, length, color, studio, studio-add, studio-class, star)∉BCNF
针对式（2）分解
STUDIO（studio, studio-add, studio-class）∈BCNF
M2（title, year, length, color, star, studio）∉BCNF
针对式（3）分解
MOVIE（title, year, length, color, studio）∈BCNF
M3（title, year, star）∈BCNF
故，将M规范化成BCNF的一个无损分解为：
ρ={STAR, STUDIO, MOVIE, M3}。
e.g. 判断题: 如果一个关系模式没有非主属性（所有属
性都是候选码的一部分），那么它至少属于 3NF。
模式分解理论
逻辑蕴涵
关系模式R(U,F)，其中U为R的属性集，F为R的函数依赖集
如果一个数据库表（关系r）完全遵守了 F 集合
里的所有规则，那么它“必然”也遵守 X → Y
这条规则
F逻辑蕴涵的所有FD称为F的闭包
Armstrom公理
分解&投影
R的一个分解
若可用R1(U1), R2(U2), … , Rk(Uk)}，U1∪U2∪…
∪Uk = U来取代原关系模式R，则称此集合为R的一个
分解。记为ρ={R1, R2, … , Rk}。
r在Ui上的投影
F在Ui上的投影
无损分解：数据无损。数据 JOIN 回去不多也不少。
任何一个关系模式都可以分解为一系列 3NF 模
式，且既具有无损连接性，又保持函数依赖。
反例
R(学生, 课程, 老师)
一个错误的分解：
R1(学生, 课程)
R2(学生, 老师)
验证（JOIN）：
R1 = { (张三, 数据库), (李四, 数据库), (张三, 操作系统) }
R2 = { (张三, 王老师), (李四, 王老师), (张三, 李老师) }
产生：
(张三, 数据库) 匹配 (张三, 王老师) -> (张三, 数据库, 王老师) (OK)
(张三, 数据库) 匹配 (张三, 李老师) -> (张三, 数据库, 李老师) (💥 虚假数据！)
保持依赖分解：规则无损。所有规则都能在小表内部被检查。
利用定义证明
模式分解规则
自反律
扩展律
传递规则
合并规则
分裂规则
结合决定性&最小性原则, 用于找到键
CHAPTER 11 DB设计
BASIS
DB设计过程
概念设计
对数据实体及其联系进行概念建模, 使用E-R or UML类图建模
视图集成法
待解问题
视图集成是合并多个视图为一个统一的全局概念
模式。合并时会遇到以下问题：
1. 对应：不同视图中存在语义等价的同一概念
2. 冲突：不同视图中存在对同一概念的不同定义
e.g.
逻辑设计
设计概念模式 & 外模式
E-R模式向关系模式的转换
转换规则
实体&属性-》创建关系模式
联系&属性-》尽量加入该联系的某个实体的关系
模式中+or为该关系单独创建关系模式
注意：
+ 模式、属性命名合理
+ 属性域（数据类型选择、定义）
+ 非原子属性（集合纵向展开，元组横向展开）
+ 选择实体键 as PK
+ 弱实体（应包含其owner实体PK）
联系转换
1:1
若两实体均非全参与，单独为联系建立关系模式
若两实体其一是全参与，则将关系属性放到该实
体的关系模式中
e.g.
1:N
考虑E2是否全参与
e.g.
M:N
特殊化转化
方案1：拆分
if d, 无数据冗余&更新异常
but 查询Ri属性时, 需连接R&Ri
方案2：合并
no need to 连接；模式个数少
but {a1, a2 ... an} 重复m次
if o，则数据冗余
if 部分特殊化，则丢失信息
范畴转化
不同类型的实体组成的新实体
为各实体定义统一键，即替身键，如“车主标识”
e.g.
注意双线表全参与
e.g. 作业题
E-R图
关系数据库模式
实体关系模式:
Student (s_id, s_name, s_gender, c_id (FK))
Class (c_id, c_name, major)
Course (course_id, course_name, credits)
Teacher (t_id, t_name, title)
联系关系模式:
Takes (s_id (FK), course_id (FK), grade)
Teaches (t_id (FK), course_id (FK))
Advises (c_id (FK), tenure_year, t_id (FK))
创建基表的 SQL DDL 语句
模式调整
改善DB性能
减少连接运算：逆规范化
将规范化程度较高的几个关系模式合并为一个较低的。
相当于提前连接好几个关系表，以减少查询时的连接运算。
缺点：导致数据冗余，易产生更新异常
不适合更新频繁的表
不常更新的表，可以通过完整性约束维护机制防
止更新异常
减小表大小：水平、垂直分割
尽量使用快照
快照snapshot是一个关系表的定期刷新的只读副本
节省存储空间
减少属性值长度：编码代替属性值、缩写代替全称
减少重复数据所占存储空间：用长度短的假属性
（dummy）值来代表长度较长的重复属性值。
物理设计
设计内模式：选择存储机制（存储结构和方
法），安排空间
目的：提高DB查询性能（main），有效利用存
储空间
技术
存储机制
无簇表
普通表
索引的表
索引的选择：PK&UNIQUE列的索引由DBMS自动建
立，其他属性列的需用户（一般DBA）建立。索引的
使用由DBMS自动决定，对用户透明
适合建立索引的情况
不适合的情况
簇表
散列簇表（常用）
簇集由用户手工建立，DBMS自动使用
索引簇表（少用）
分区设计：数据合理分布多个磁盘，以合理安排
I/O，提高DB效率
水平分割大的关系，分散至多个磁盘阵列中。
可减少访盘冲突，提高I/O并行性
使热点数据分散
可均衡I/O负荷
保证关键数据（如DD）快速访问
缓解系统访问瓶颈
站在数据库设计者角度
Part of
统称一般性约束
图例
Not important
重要内容
Like
补充
SUM速览
25年考试内容
Question